Universidad Politécnica de Cartagena

MEMBER OF EUT+ EUROPEAN UNIVERSITY OF TECHNOLOGY

# PINZA ACCIONADA POR SENSOR ECG

## Fundamentos de Electrónica Industrial

# OBJETIVOS Y ASPECTOS BÁSICOS

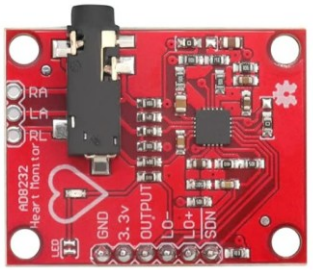
El objetivo de este proyecto es crear un sistema que permita a personas con problemas de movilidad en las manos poder coger objetos simples.

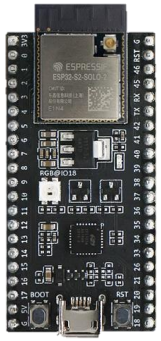
Para ello necesitamos:

1. Obtener valores de actividad de la musculatura del antebrazo.
2. Procesar los valores para actuar conforme a las variaciones.
3. Accionar un motor a través de un algoritmo.
4. Mostrar datos al usuario.
5. Permitir el control.

# MATERIALES

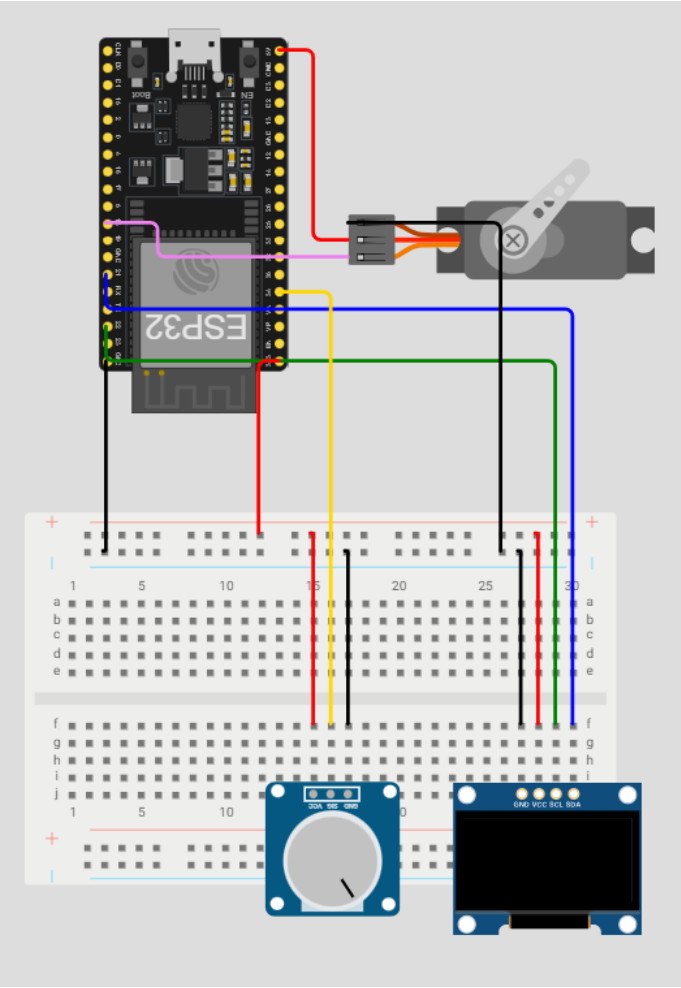
| COMPONENTE | FUNCIÓN | PRECIO ESTIMADO |
|---|---|---|
| ESP-32D (Dev Kit) | Microcontrolador | 3€ |
| Sensor AD8232 | Obtención datos | 4,5€ |
| Servo Motor 9G MS18 | Movimiento pinza | 2,2€ |
| Pantalla OLED I2C | Información al usuario | 1,8€ |
| Protoboard | Prototipado | 8€ |
| *Terminal Adapter* | Prototipado | 2.5€ |
| Cables | Montaje | 5€ |



Universidad Politécnica de Cartagena
MEMBER OF
EUROPEAN UNIVERSITY OF TECHNOLOGY

# ESQUEMA DE CONEXIONES



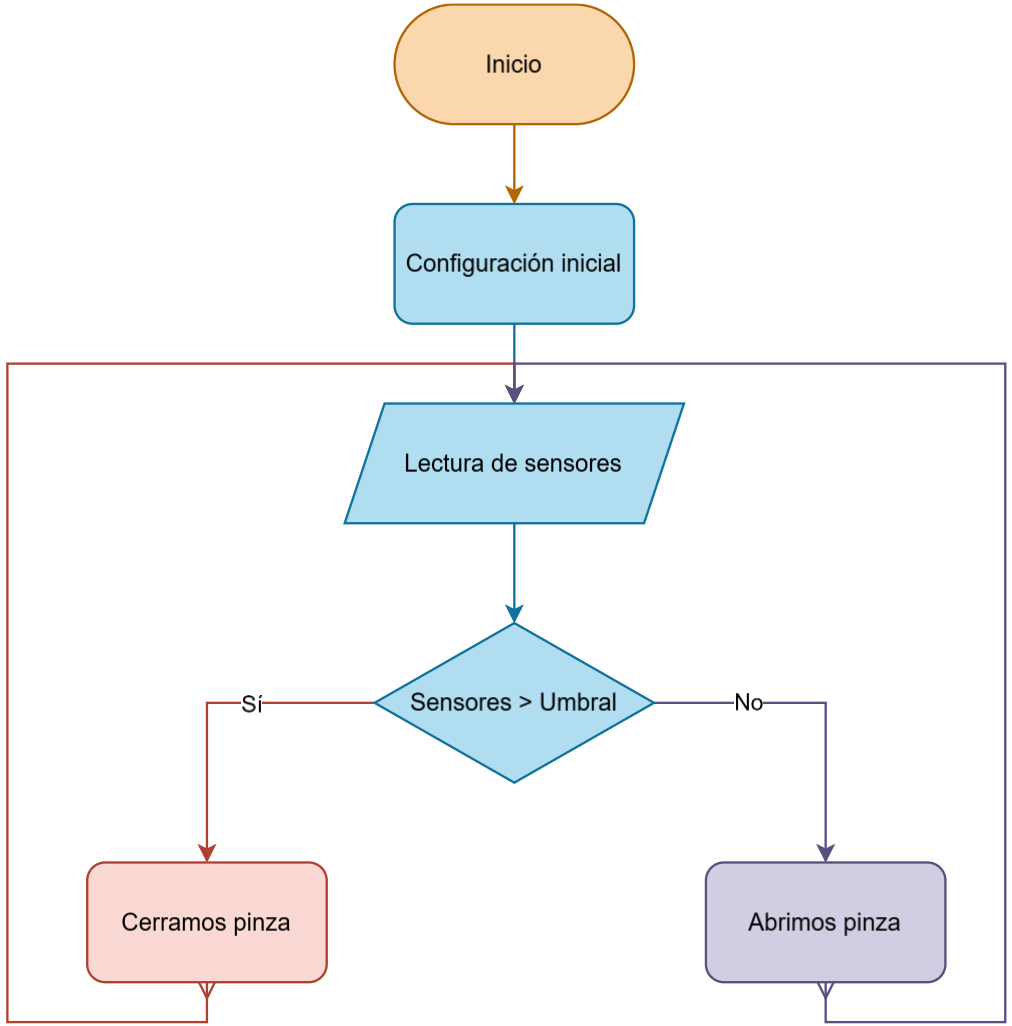

**GUÍA DE CONEXIONES:**

| | | |
|---|---|---|
| Pin SCL OLED | → | 22 |
| Pin SDA OLED | → | 21 |
| Pin AD8232 | → | 34 |
| Pin Servo | → | 18 |

**IMPORTANTE:**

- El Servo necesita alimentación de 5V
- Necesitamos tener un GND común entre la fuente de alimentación del Servo y el ESP32 para permitir el funcionamiento del PWM.

# LÓGICA Y CÓDIGO






Universidad Politécnica de Cartagena
MEMBER OF
EUROPEAN UNIVERSITY OF TECHNOLOGY

# LÓGICA Y CÓDIGO

1. Configuración: librerías y pines

```
#define BLYNK_TEMPLATE_ID "#############"
#define BLYNK_TEMPLATE_NAME "ESP32 - PINZA MÚSCULO"
#define BLYNK_AUTH_TOKEN "##############################"

// Librerías y códigos a importar:
#include <ESP32Servo.h>        //Servo
#include <Wire.h>              // Para el protocolo I2C de la pantalla
#include <Adafruit_GFX.h>      // Pantalla
#include <Adafruit_SSD1306.h>  // Pantalla
#include <WiFi.h>              // Wifi
#include <WiFiClient.h>        // Wifi
#include <BlynkSimpleEsp32.h>  //Blynk

// Imagenes
#include "Brazo.h"
```

```
// Puertos
#define pinServo 18
#define musculoECG 34
// SCL 22
//SDA 21
#define LED 2

// Propiedades pantalla
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

#define BLYNK_PRINT Serial  // Para DEBUG, quitar cuando no haga falta
```

# LÓGICA Y CÓDIGO

1. Configuración: objetos y variables

```
Servo servo;
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
BlynkTimer timer;


// Establecemos las variables a usar de forma global
char ssid[] = "#######################";  // cambiar por SSID
char pass[] = "########";
unsigned long mili;
int encendido;
```

```
// Filtro de media movil
float musculo = 0;
float MUS_filtrado = 0;
float alpha = 0.05;  // Valor de filtrado

unsigned long t_lectura_ant;
int n_lectura = 50;
```

# LÓGICA Y CÓDIGO

1. Funciones personalizadas

```
// Funciones eprsonalizadas
void textoPantalla(String texto, bool scroll) {
  display.clearDisplay();
  display.setTextSize(2);
  display.setTextColor(WHITE);
  display.setCursor(0, 10);
  display.println(texto);
  display.display();
  if (scroll) {
    display.startscrollright(0x00, 0x0F);
  } else {
    display.stopscroll();
  }

  // Mostramos el texto tambien en la web y app
  Blynk.virtualWrite(V5, texto);
}
```

```
void myTimerEvent()  // Sacado de documentation
{
  Blynk.virtualWrite(V2, millis() / 1000);
}


BLYNK_WRITE(V0) {
  encendido = param.asInt();
  display.clearDisplay();
  display.drawBitmap(0, 0, image_data_Brazo, 128, 64, 1);
  display.display();
  digitalWrite(LED, LOW);
  Serial.println("Cambio en estado del sistema.");
}
```

# LÓGICA Y CÓDIGO

1. Configuración

```
void setup() {
  Serial.begin(115200);  // Para poder usar de consola.

  // Iniciamos la pantalla
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println(F("Error al conectar la pantalla.\n Recuerde pines:\n\t SCL: 22 \n\t SDA: 21"));
    pinMode(LED, OUTPUT);
    while(true){
      digitalWrite(LED, HIGH);
      delay(200); // Como ha habido un error podemos usar un delay
      digitalWrite(LED, LOW);
      delay(200);
    }
  }

  textoPantalla("Iniciando", false);


  pinMode(pinServo, OUTPUT);
  servo.attach(pinServo);
  servo.write(45);  // Ponemos la pinza en abierto como estado por defecto.

  pinMode(musculoECG, INPUT);

  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
  timer.setInterval(1000L, myTimerEvent);

  Blynk.virtualWrite(V4, 0);
  Blynk.virtualWrite(V7, 1);
  encendido = 1;

  display.clearDisplay();
  display.display();

  Blynk.virtualWrite(V4, 255);  // Para mostrar verde en web

  pinMode(LED, OUTPUT);
```

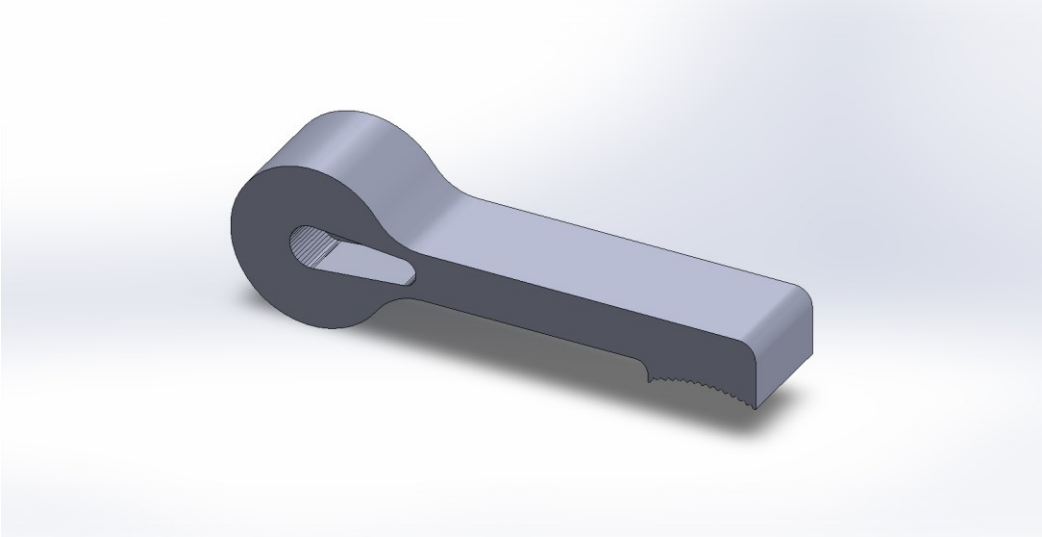
# LÓGICA Y CÓDIGO

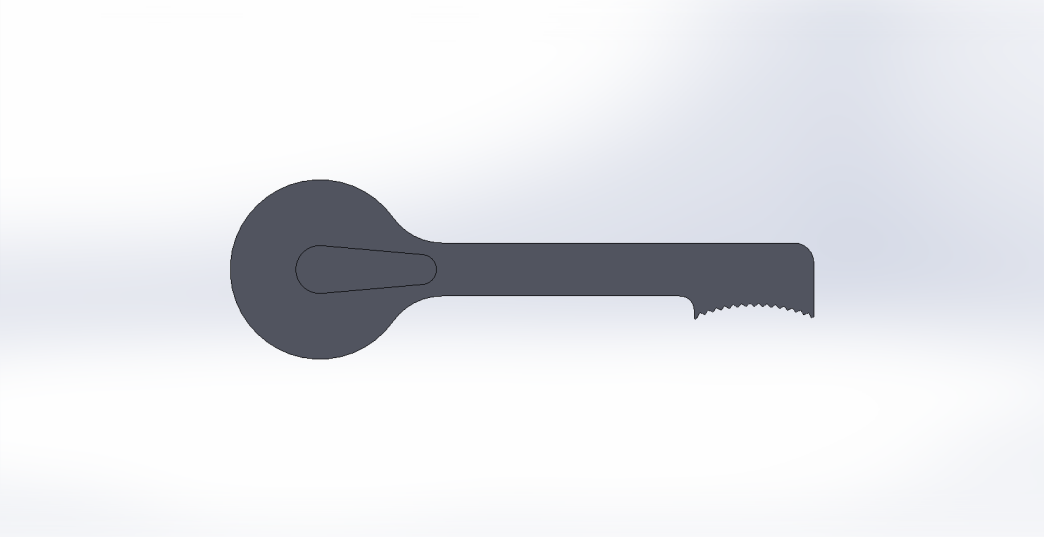
1. *Loop*: lógica y funcionamiento principal

```
void loop() {

  Blynk.run();


  timer.run();  // Para poner el Uptime en la web.

  //musculo = random(1, 4095);  // Quitar para uso en producción
  // Si está encendido medimos y hacemos media movil, aunque no movamos la pinza, para evitar escalones
  if (millis() - t_lectura_ant > n_lectura && encendido == 1) {
    t_lectura_ant = millis();
    // Obtenemos los datos del sensor del músculo
    musculo = analogRead(musculoECG);
    MUS_filtrado = (alpha * musculo) + ((1.0 - alpha) * MUS_filtrado);

    Serial.println(String(musculo) + "," + String(MUS_filtrado, 1));  // Para Excel (no va si el resto
está activado)
    digitalWrite(LED, LOW);
  }

  //Lógica pinza
  if ((millis() - mili) >= 500 && encendido == 1) {  // Ejecutamos cada 0.5s
    mili = millis();
    digitalWrite(LED, HIGH);
    Blynk.run();
    Blynk.virtualWrite(V7, MUS_filtrado);
    Blynk.virtualWrite(V8, musculo);  // Enviamos valores cada 0.5s para evitar usar muchas llamadas.
    // Lógica pinza
    if (MUS_filtrado >= 2047) {  // Si detectamos fuersza cerramos.
      servo.write(90);
      Serial.println("Pinza cerrada, detectado valor superior a 2047 en sensor");
      textoPantalla("Cerrada\nHaciendo\nfuerza.", false);


    } else {
      // Si no detectamos, abrimos.
      servo.write(45);
      Serial.println("Pinza abierta.");
      textoPantalla("Abierta.", false);
    }
  }
}
```

# DISEÑO 3D DE PIEZAS

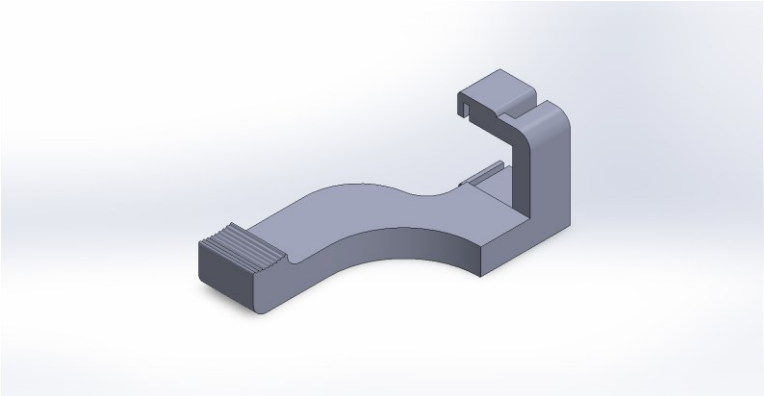
## Pieza 1

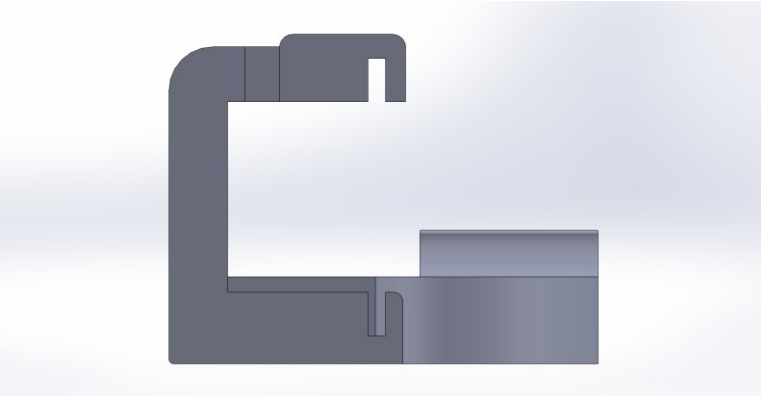


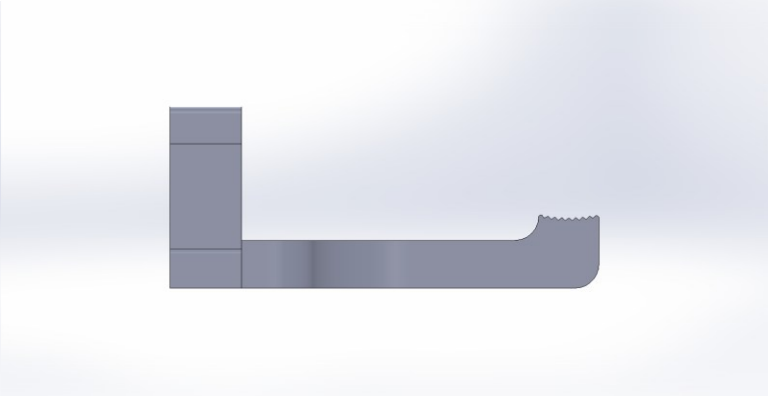


# DISEÑO 3D DE PIEZAS

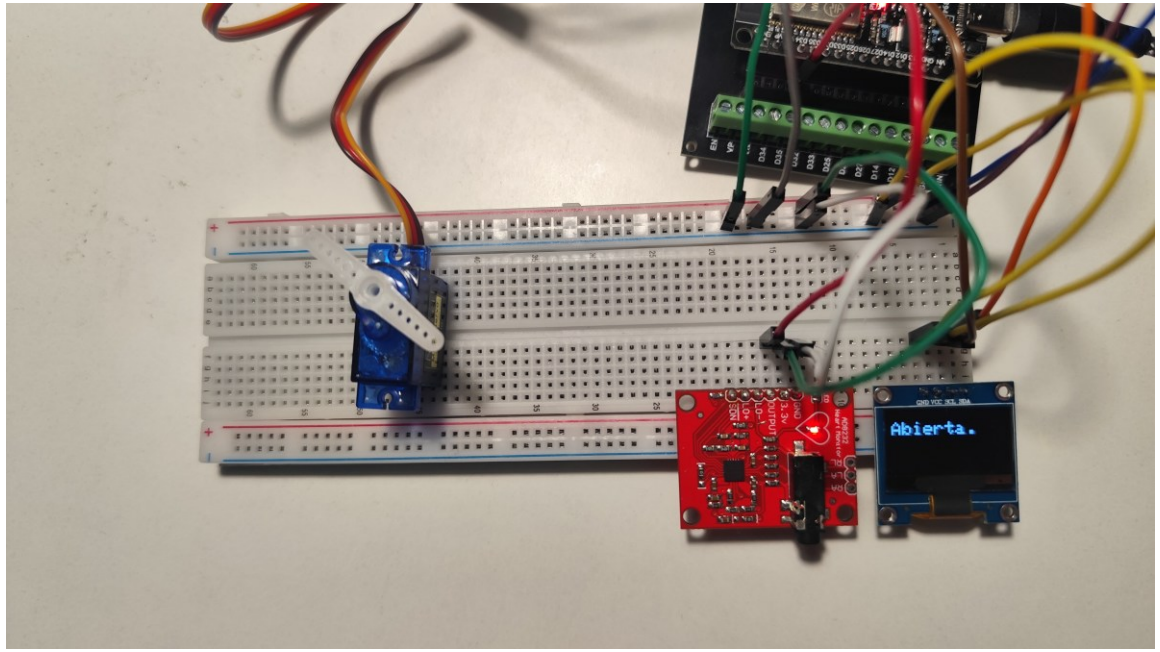
## Pieza 2

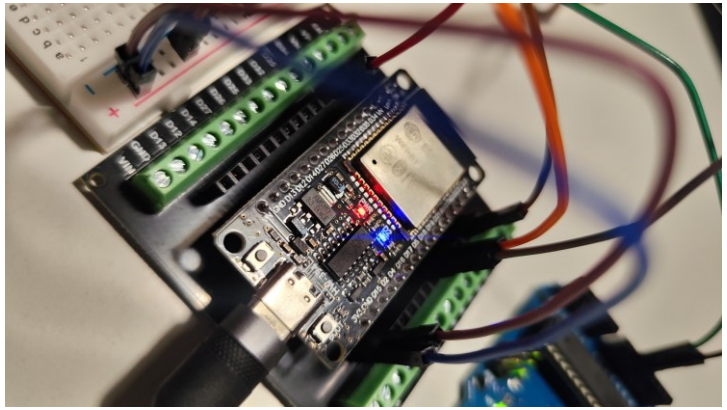


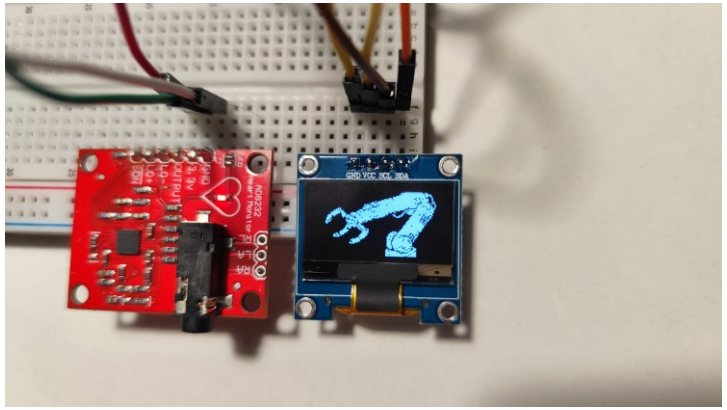




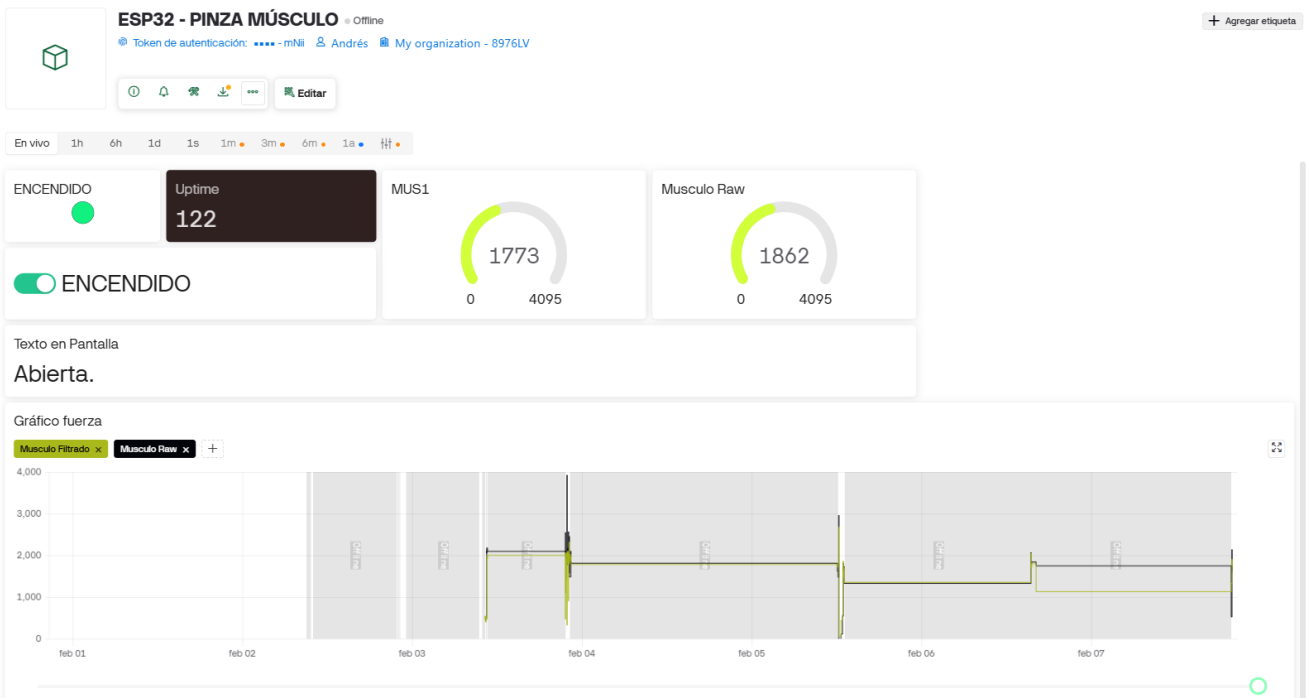
# FOTOGRAFÍA





# *INTEGRACIÓN IoT*

**B BLYNK**

Integración en plataforma *Low-code* de IoT para control remoto y visualización de datos